

CloudKernel

Cloud Hypervisor Simulation — Project Report & Manual

Java Multithreading & Synchronization | Operating Systems Lab

CountDownLatch

CyclicBarrier

Semaphore

Java Swing GUI

NFC Institute of Engineering & Technology, Multan
Department of Artificial Intelligence | BSAI-2k24
Submitted: March 03, 2026 | 4th Semester

Submitted By

Moavia Amir	2k24_BSAI_72
Ali Raza	2k24_BSAI_44
Muhammad Arslan Nasir	2k24_BSAI_26

Submitted To

Mam Amara Nadeem	Operating Systems Lab
------------------	-----------------------

Chapter 1 — What is CloudKernel?

1.1 The Big Picture — In Simple Words

Imagine a huge, powerful computer in a data centre. Companies like Amazon (AWS) and Google Cloud use such machines to run hundreds of smaller, virtual computers inside a single physical one. Each of these smaller computers is called a **Virtual Machine (VM)**.

Something needs to manage all these VMs — making sure they do not fight over the same resources, stay in step with each other, and share things fairly. That manager is called a **Hypervisor**.

CloudKernel is a simplified, educational version of a hypervisor, written in Java. It does not use real hardware. Instead, it uses **Java Threads** to pretend to be VMs, and **Java synchronization tools** to manage them — the same concepts used in real systems like KVM and VMware.

1.2 Why Was It Built?

CloudKernel was created for an Operating Systems Lab assignment to show, in observable and readable code, how an OS handles three classic problems:

- Making sure the system is ready before anything starts.
- Keeping multiple processes in sync with each other.
- Sharing limited resources fairly without crashes or deadlocks.

1.3 What Language and Tools?

Language	Java (java.util.concurrent library)
GUI Framework	Java Swing
Key Classes Used	CountDownLatch, CyclicBarrier, Semaphore, Thread
IDE Recommended	IntelliJ IDEA / VS Code with Java Extension
How to Run	<code>javac -d out -sourcepath src src/Main.java</code> then <code>java -cp out Main</code>

Chapter 2 — The Three Core Concepts

CloudKernel is built around three Java synchronization tools. Each one solves a different real-world problem. This chapter explains each one in plain words.

2.1 CountdownLatch — The Boot Gate

What problem does it solve?

Before any VM can start running, the system must finish setting up shared resources like Disk, RAM, Network Stack, and CPU Scheduler. If a VM starts too early, it might try to use something that is not ready yet — causing crashes.

How does CountdownLatch fix this?

Think of it like a rocket launch countdown. The latch starts at a number (4 in CloudKernel — one for each resource). Every time a resource finishes initializing, it calls **countDown()**, reducing the number by 1. All VMs call **await()** and are blocked (frozen) until the counter reaches zero. Once it hits zero, everyone is released at the same time.

```
CountDownLatch latch = new CountDownLatch(4); // Each resource calls:  
latch.countDown(); // Each VM calls: latch.await(); // waits here until count = 0
```

Note: CountdownLatch can only count down — it cannot reset. Once it reaches zero it stays open forever.

2.2 CyclicBarrier — The Sync Checkpoint

What problem does it solve?

All 5 VMs work in rounds called cycles. The rule is: no VM can start the next cycle until every VM has finished the current one. Without this, fast VMs would race ahead and slow ones would fall behind — making the system unpredictable.

How does CyclicBarrier fix this?

Think of it like a group of hikers who agree to wait at a checkpoint before continuing. Each VM does its work for the cycle, then calls **await()** on the barrier. Once all 5 VMs have arrived at the checkpoint, the barrier opens and everyone moves to the next cycle together.

```
CyclicBarrier barrier = new CyclicBarrier(5); // Each VM calls at end of cycle:  
barrier.await(); // When all 5 call it, they are all released together
```

Note: Unlike CountdownLatch, CyclicBarrier automatically resets after each cycle — that is why it is called Cyclic.

2.3 Semaphore — The Resource Gatekeeper

What problem does it solve?

There are limited shared resources — only 2 Network Ports, 2 Memory Blocks, and 3 CPU Cores. If all 5 VMs grab the same resource at once, the system would be overwhelmed. We need a fair queuing system.

How does Semaphore fix this?

A Semaphore holds a certain number of permits (like tokens). To use a resource, a VM must acquire a permit. If no permits are available, the VM waits in a queue. When it is done, it releases the permit so the next waiting VM can have it.

```
Semaphore networkSem = new Semaphore(2); networkSem.acquire(); // grab a permit –  
waits if none free // ... use the network ... networkSem.release(); // return the  
permit for others
```

CloudKernel also uses **tryAcquire(2, TimeUnit.SECONDS)** — if a VM cannot get a permit within 2 seconds, it gives up and logs a TIMEOUT instead of waiting forever. This prevents deadlocks.

Chapter 3 — Project Structure

The project is organized into clean packages so each file has one clear job.

Package / File	What it does
src/Main.java	Entry point — launches the GUI
core/BootManager.java	Manages CountdownLatch, initializes resources
core/ClockSynchronizer.java	Manages CyclicBarrier, syncs VM cycles
core/NetworkPortManager.java	Manages Semaphores for all shared resources
entities/VirtualMachine.java	The VM thread — runs tasks, requests resources
entities/VMState.java	Enum of all possible VM states
entities/VMStats.java	Tracks each VM's stats (tasks, timeouts, waits)
config/SimulationConfig.java	Reads config.properties settings
utils/Logger.java	Prints color-coded logs to terminal
utils/StatsCollector.java	Gathers system-wide stats for final report
shutdown/ShutdownManager.java	Handles Ctrl+C graceful shutdown
ui/CloudKernelGUI.java	Main GUI window
ui/VMCard.java	One VM card panel in the dashboard
ui/ResourceMonitorPanel.java	Left sidebar showing semaphore slots
ui/BarrierPanel.java	CyclicBarrier visualizer panel
ui/LogPanel.java	Color-coded live log panel
ui/StatsBar.java	Bottom statistics bar
ui/ControlPanel.java	Start / Pause / Reset buttons and speed slider

Chapter 4 — VM Lifecycle States

Every VM goes through a clear sequence of states during the simulation. Understanding these states makes it easy to follow what is happening in both the terminal logs and the GUI dashboard.

State	What it means
BOOTING	VM is waiting for the CountdownLatch to reach zero (system not ready yet)
READY	System is booted, VM is about to start its first cycle
RUNNING	VM is actively executing tasks in the current cycle
REQUESTING	VM is trying to acquire a semaphore permit for a resource
USING RESOURCE	VM holds a permit and is using CPU / Memory / Network
RELEASING	VM is releasing the permit back to the semaphore
BARRIER WAIT	VM finished its cycle and is waiting at the CyclicBarrier for others
TIMEOUT	VM waited too long for a resource and gave up (tryAcquire expired)
SHUTDOWN	Simulation ended, VM thread is stopping cleanly

State Flow Diagram (simplified)

```

BOOTING | v (CountDownLatch reaches 0) READY | v RUNNING --> REQUESTING --> USING
RESOURCE --> RELEASING | |
|<-----<| | v (end of cycle)
BARRIER WAIT | v (all VMs arrive) RUNNING (next cycle starts) | v (all cycles done)
SHUTDOWN

```

Chapter 5 — The GUI Dashboard (Full Walkthrough)

The GUI is built using **Java Swing**. It looks like a real cloud server monitoring dashboard — dark background, cyan accents, monospace font, and live updating panels. Every panel directly visualizes one of the concurrency concepts.

Note: All GUI updates from background threads are done using `SwingUtilities.invokeLater()` to keep the interface safe and responsive.

5.1 Main Window

Window Title	CloudKernel — Cloud Hypervisor Monitor
Size	1280 x 800 pixels, centered on screen
Background Color	#0a0e1a (very dark navy blue)
Text Color	#c8d8f0 (light blue-white)
Accent Color	#00d4ff (bright cyan)
Font	JetBrains Mono or Consolas — monospace, like a terminal
Layout	BorderLayout — header top, sidebar left, log right, stats bottom

5.2 Header Bar (Top Strip)

The header bar runs across the top of the window and shows three things:

- **Project title on the left:**  CLOUDKERNEL HYPERVISOR' in bold cyan text.
- **Live digital clock on the right:** Updates every second using `javax.swing.Timer`.
- **System status indicator:** A green dot  SYSTEM ONLINE' that blinks every 500ms after the system boots. Before boot it is gray and says 'OFFLINE'.

5.3 Boot Manager Panel — CountdownLatch Visualized

This panel sits just below the header. It makes the `CountDownLatch` countdown visible to the user.

- **4 resource chips** are shown in a row: Disk, RAM, Network Stack, CPU Scheduler.
- Each chip starts gray (`#1e2d45`). When the resource finishes initializing it turns glowing green (`#00ff88`) with a shadow effect.
- **Latch counter** below the chips counts down visibly: '4 → 3 → 2 → 1 → 0 (RELEASED)'.
- When the latch hits zero, a banner flashes: 'ALL SYSTEMS READY — VMs STARTING'.

```
// In Java Swing (simplified):
resourceChip.setBackground(Color.decode("#00ff88")); // turn green
```

```
latchLabel.setText("LATCH: " + remaining); // update count
```

5.4 VM Dashboard Panel — 5 VM Cards (Center)

This is the main area of the GUI. It shows 5 VM cards side by side using `GridLayout(1, 5)`. Each card is a `JPanel` with a dark border.

Inside each VM card:

- **VM Name** at the top in bold white — VM-1, VM-2, VM-3, VM-4, VM-5.
- **Priority badge**: HIGH in red, MEDIUM in amber, LOW in cyan — small colored label.
- **State label**: Shows the current lifecycle state with matching color (cyan=RUNNING, green=USING RESOURCE, amber=WAITING, purple=BARRIER WAIT, red=TIMEOUT).
- **Progress bar**: A `JProgressBar` styled dark, filled in cyan, showing how far through the current cycle the VM is (0% to 100%).
- **Resource dots**: Three small colored circles — one each for CPU, Memory, Network. Lit up when the VM currently holds that resource.
- **Task counter**: 'Tasks: 12' — total tasks completed so far.
- **Average wait time**: 'Avg Wait: 320ms' — average time spent waiting for resources.

Card border color changes with state:

RUNNING	Cyan border (#00d4ff)
USING RESOURCE (Network)	Green border (#00ff88)
REQUESTING / WAITING	Amber border (#ffbb00)
BARRIER WAIT	Purple border (#bb88ff)
TIMEOUT	Red border (#ff4466)
BOOTING / IDLE	Default dark border (#1e2d45)

5.5 Resource Monitor Panel — Semaphores Visualized (Left Sidebar)

This panel on the left side of the window shows the real-time state of all three Semaphores. This is one of the most important panels for understanding how resource sharing works.

Three sections, one per resource type:

- **CPU Cores — Semaphore(3)**: Shows 3 slot boxes. Each box is green with 'VM-X' when a VM holds it, or gray with 'FREE' when available.
- **Memory Blocks — Semaphore(2)**: Shows 2 slot boxes. Same color logic.
- **Network Ports — Semaphore(2)**: Shows 2 slot boxes. Same color logic.

Every time a VM acquires or releases a semaphore permit, the panel updates instantly via `SwingUtilities.invokeLater()`. The permit count is shown as 'Permits: 1/2 available'.


```
// When VM acquires network semaphore:
semSlot[0].setBackground(Color.decode("#00ff88")); semSlot[0].setText("VM-" +
vmId); // When VM releases: semSlot[0].setBackground(Color.decode("#1e2d45"));
semSlot[0].setText("FREE");
```

5.6 CyclicBarrier Visualizer Panel

This panel sits below the VM cards and shows the barrier synchronization in a visual and easy-to-understand way.

- **5 VM dot indicators** in a row — one per VM.
- Each dot is dark gray by default. When a VM finishes its cycle and calls `barrier.await()`, its dot **turns purple**.
- When all 5 dots are purple (all VMs arrived), the barrier opens — all dots **flash green briefly** using a Swing Timer animation, then reset to gray.
- **Cycle counter** updates: 'CYCLE: 4'
- **Arrived counter** updates live: 'Arrived: 3 / 5'

Note: The flash animation is created using `javax.swing.Timer` with a 400ms delay — it fires once, turns all dots green, then resets them after another 400ms.

5.7 Live Log Panel (Right Sidebar)

A JTextPane on the right side that shows every event in real time with color-coded text so different types of messages are easy to spot.

Log Type	Color	Example
[BOOT]	Cyan	[00:01.2] [BOOT] Disk initialized
[NETWORK] acquire	Green	[00:03.4] [NETWORK] VM-2 acquired port 1
[WAITING]	Amber	[00:03.4] [SEMAPHORE] VM-4 waiting for CPU core
[BARRIER]	Purple	[00:05.1] [BARRIER] VM-1 arrived at checkpoint
[TIMEOUT]	Red	[00:06.8] [TIMEOUT] VM-3 gave up waiting for Memory
[VM] general	White	[00:04.0] [VM-2] Completed task #7
[CYCLE] complete	Amber	[00:07.0] [SYSTEM] Cycle 2 complete — all VMs released

The log auto-scrolls to the bottom on every new entry. Old entries are kept so the full history is readable by scrolling up.

5.8 Statistics Bar (Bottom Strip)

A thin dark bar at the very bottom showing live system statistics as small cards:

Total Cycles	How many full cycles all VMs have completed
Network Ops	Total number of successful network semaphore acquisitions
CPU Ops	Total CPU core semaphore acquisitions
Timeouts	Total number of tryAcquire timeouts across all VMs
Contentions	Times a VM had to wait because all permits were taken
Uptime	Time since system boot in MM:SS format

5.9 Control Panel (Bottom Bar)

■ BOOT SYSTEM button	Starts the simulation — initializes resources then launches VM threads
■ PAUSE button	Pauses all VM threads by setting a shared volatile boolean flag
■ RESET button	Stops all threads, clears all panels, resets stats to zero
Speed slider (0.5x–3x)	Adjusts Thread.sleep() durations — higher speed = shorter sleeps

All buttons are styled with dark background, colored borders (cyan/red), JetBrains Mono font, and a subtle hover effect that lightens the background.

Chapter 6 — How the GUI and Threads Work Together

This is an important chapter. The GUI and the background threads (VMs) run in parallel. They must communicate safely without causing visual glitches or crashes. Here is exactly how CloudKernel does it.

6.1 The Golden Rule — `SwingUtilities.invokeLater()`

Java Swing is not thread-safe. Only one thread — the **Event Dispatch Thread (EDT)** — is allowed to update the GUI. VM threads run on separate background threads. So whenever a VM wants to update a panel, it must use:

```
SwingUtilities.invokeLater(() -> { vmCard.updateState(VMState.USING_RESOURCE);  
resourcePanel.setSlotOccupied(0, vmId); });
```

This schedules the update to run on the EDT safely, without interrupting the VM thread.

6.2 GUILogger — Logging to Both Terminal and GUI

The GUILogger class replaces the simple Logger. Every log call goes to both the terminal (with ANSI colors) and the GUI log panel (with Swing colors) at the same time.

```
public class GUILogger { public static void log(String msg, LogType type) {  
System.out.println(ANSI_COLOR + msg); // terminal SwingUtilities.invokeLater(() ->  
// GUI logPanel.appendColored(msg, type) ); } }
```

6.3 DashboardUpdater — Keeping All Panels in Sync

The DashboardUpdater class is the bridge between VM threads and GUI panels. It has one update method that receives a VM state change event and refreshes every relevant panel:

```
public class DashboardUpdater { public void onStateChange(int vmId, VMState  
newState) { SwingUtilities.invokeLater(() -> { vmCards[vmId].setState(newState);  
if (newState == VMState.USING_RESOURCE) resourcePanel.markOccupied(vmId); if  
(newState == VMState.BARRIER_WAIT) barrierPanel.markArrived(vmId);  
statsBar.refresh(); }); } }
```

6.4 Swing Timers for Animation

`javax.swing.Timer` is used for three GUI animations — it is safe to use from the EDT unlike `java.util.Timer`:

- **Live clock:** Fires every 1000ms, updates the time display in the header.
- **Blinking status dot:** Fires every 500ms, toggles the green dot between bright green and dark green.

- **Barrier flash:** When all VMs arrive, a one-shot timer fires after 400ms to flash all barrier dots green, then resets them after another 400ms.

```
javax.swing.Timer clockTimer = new javax.swing.Timer(1000, e -> {  
clockLabel.setText(LocalTime.now().format(formatter)); }); clockTimer.start();
```

Chapter 7 — Configuration & Graceful Shutdown

7.1 config.properties — Customizing the Simulation

At startup, CloudKernel reads a file called config.properties. You can change any of these settings without touching the code:

```
# Number of virtual machines to simulate vm.count=5 # How many cycles each VM runs
cycle.count=4 # Max VMs allowed to use Network at the same time semaphore.network=2
# Max VMs allowed to use Memory at the same time semaphore.memory=2 # Max VMs
allowed to use CPU at the same time semaphore.cpu=3 # How long each task takes
(millisecons) task.duration.min=500 task.duration.max=1500 # How long a VM waits
before giving up on a resource (timeout) timeout.ms=2000
```

If the file is missing, CloudKernel uses built-in defaults and prints a yellow warning in the log.

7.2 Graceful Shutdown — Ctrl+C Handler

If you press Ctrl+C while the simulation is running, CloudKernel does not crash. A ShutdownManager catches the signal and:

- Prints a partial summary of all cycles completed so far.
- Releases all semaphore permits so no thread is stuck waiting.
- Interrupts all VM threads cleanly.
- Saves final stats to a log file.

```
Runtime.getRuntime().addShutdownHook(new Thread(() -> {
ShutdownManager.handleShutdown(); }));
```

Chapter 8 — Final Statistics Report

After all cycles complete (or on shutdown), CloudKernel prints a clean summary table in both the terminal and the GUI.

8.1 Per-VM Statistics Table

VM	Tasks Done	Net Uses	CPU Uses	Timeouts	Avg Wait
VM-1 (HIGH)	24	8	10	1	312ms
VM-2 (MEDIUM)	21	7	9	2	445ms
VM-3 (LOW)	18	6	8	4	680ms
VM-4 (HIGH)	23	9	11	0	290ms
VM-5 (MEDIUM)	20	7	9	3	510ms

8.2 System-Wide Summary

Total Cycles Completed	4
Total Semaphore Contentions	14
Total Timeouts	10
Total Network Operations	37
Total CPU Operations	47
Peak Concurrent VM Activity	5 VMs running simultaneously
Total Simulation Time	42 seconds

Chapter 9 — How to Run the Project

Step 1 — Compile

```
javac -d out -sourcepath src src/Main.java
```

Step 2 — Run

```
java -cp out Main
```

Step 3 — Edit settings (optional)

```
# Open config.properties and change any values, then run again
```

Step 4 — Use the GUI

- Click **BOOT SYSTEM** to start the simulation.
- Watch the Boot Panel chips turn green one by one as resources initialize.
- Watch the 5 VM cards change color and state as they run.
- Watch the Resource Monitor slots fill up and free up as VMs share resources.
- Watch the Barrier dots turn purple as VMs finish each cycle.
- Read the live log on the right to follow every event.
- Click **PAUSE** to freeze everything, then resume.
- Click **RESET** to start over from scratch.

Troubleshooting

GUI does not open	Make sure you are using Java 11 or higher
Font looks wrong	Install JetBrains Mono or the GUI falls back to Consolas
config.properties not found	A warning is printed and defaults are used — this is fine
Threads not stopping	Click Reset — it interrupts all threads cleanly

Chapter 10 — Summary & What We Learned

CloudKernel started as a simple terminal simulation and grew into a full professional dashboard application. Here is a quick recap of everything covered:

Concept	Java Tool	Where Visible in GUI
System must be ready before VMs start	CountDownLatch	Boot Manager Panel — chips turn green
All VMs must finish a cycle before next begins	CyclicBarrier	Barrier Panel — dots turn purple then flash green
Only limited VMs can use a resource at once	Semaphore	Resource Monitor — slots show FREE or VM-X
GUI updates safely from background threads	SwingUtilities.invokeLater()	Every panel in the dashboard
Animations without freezing the UI	javax.swing.Timer	Clock, blinking dot, barrier flash
Avoid deadlocks on resource wait	waitAcquire(timeout)	TIMEOUT state shown in red on VM cards

This project demonstrates that the concepts taught in Operating Systems — synchronization, resource sharing, process management — are not just theory. They are the foundation of every cloud platform running today.